

TRUPEST — SYSTEM REQUIREMENTS SPECIFICATION (SRS) v2.6

Format: B (Engineering Specification)

Status: FINAL RELEASE (Production Ready + Timing & Latency Hardening)

Version: 2.6

Target Audience: Lead Engineer, Backend Developer, QA Engineer, DevOps, AI Scientist

1. INTRODUCTION

1.1 Purpose

Dokumen ini mendefinisikan struktur sistem, kebutuhan teknis, arsitektur, WebSocket API, model data, mekanisme recovery, validasi server, AI inference pipeline, dan anti-cheat enforcement untuk proyek **TRUPEST**.

Dokumen ini adalah **dasar implementasi mutlak**. Setiap deviasi kode dari dokumen ini dianggap sebagai **defect**. Update v2.6 menyuntikkan **Game Tick Architecture**, **Turn Action Timer**, dan **Latency/Heartbeat Protocols** untuk stabilitas jaringan.

1.2 Scope

Mencakup:

- **Core**: Game Server Logic, State Machine, & Rules Enforcement.
- **Timing & Scheduling**: Server Tick Rate (20Hz), Turn Timer, & Freeze Timeout (**Updated v2.6**).
- **Security**: Client Lockdown (No Client-Side Logic), Anti-Cheat, & Telemetry.
- **Network**: Heartbeat, Latency Tolerance, & Resync Logic (**Updated v2.6**).
- **Deterministic Replay**: Event-Only Replay Engine.
- **Matchmaking**: Algorithm & MMR System.
- **AI**: Logic Tiers & Inference Pipeline w/ Metadata (**Updated v2.6**).
- **Infrastructure**: Database Schema, ETL, & Persistence.

Tidak Mencakup:

- Desain UI Final (Lihat GDD Format A).
- Monetisasi Store (Phase 2).
- Admin Tools (Phase 2).

1.3 Definitions

- **Trick**: 1 putaran permainan di mana 4 kartu dimainkan.
- **Declarer**: Pemain yang memenangkan fase bidding.
- **Hidden Trump**: Kartu trump yang dimainkan secara tertutup dan nilainya disembunyikan server hingga trick selesai.

- **AI Takeover:** Mekanisme bot otomatis menggantikan pemain karena inaktivitas/disconnect.
- **Snapshot:** State permainan yang diserialisasi ke Redis untuk recovery.
- **Replay Engine:** Modul di client/server yang merekonstruksi match hanya dari event log.
- **newAccount:** Akun dengan umur < 3 hari ATAU totalMatch < 20.
- **Tick:** Satu siklus update logika server (Standard: 50ms).

2. SYSTEM OVERVIEW

2.1 Architecture

- **Model:** Server-Authoritative (**Client is Dumb Terminal / UI-Only**).
- **Frontend:** HTML5/React → WebSocket.
 - **Constraint:** Client tidak boleh menyimpan rule game. Semua validasi, penentuan main suit, dan perhitungan skor dilakukan di server.
- **Backend:** NodeJS/TypeScript (Recommended) atau Go.
- **Database (Persistent):** PostgreSQL (Match logs, User profiles).
- **Database (Hot/Session):** Redis (Session storage, Game State, Pub/Sub, Streams).
- **Backup:** Snapshot state ke Redis setiap akhir **Trick**.

2.2 High-Level Data Flow

[Client UI] ↑ (WebSocket w/ JWT) [Load Balancer] ↑ (RPC) [Game Server] ↑ (Write-Through)
[Redis] ↓ (Async ETL) [PostgreSQL]

3. FUNCTIONAL REQUIREMENTS

3.1 Player Authentication

- Mendukung OAuth atau Username/Password.
- Client wajib menyertakan **Session Token** (JWT) pada handshake WebSocket.
- Server memvalidasi token dan mencegah **double-login** (kick sesi lama).

3.2 Room System

- **Create Room:** User dapat membuat room Public atau Private.
- **Seat Randomizer:** Posisi duduk diacak server saat game start.
- **Kick Logic:**
 - **Inactivity:** Kick otomatis jika idle > 60 detik di lobby.
 - **Vote Kick:** Butuh 3/4 suara (Hanya di Custom Room).

3.3 Match Lifecycle (Updated v2.5)

1. **Init:** Room ready (4 players).
2. **Setup:** Shuffle (HMAC Seed) → Deal Cards.
3. **Dealer Assignment** (Deterministic):
 - roundSeed = HMAC_SHA256(serverSeed, matchId || ":" || roundNumber)

- $\text{dealerIndex} = \text{roundSeed}[0] \% 4$
- 4. **Phase 1:** Bidding.
- 5. **Phase 2:** Trump Selection.
- 6. **Phase 3:** Trick Loop (1–13).
- 7. **Phase 4:** Scoring & Evaluation.
- 8. **Phase 5:** Dealer Rotation / Ready Prompt / End Match.

4. GAME LOGIC REQUIREMENTS

4.1 Shuffle (Provably Fair & Deterministic)

- **Requirement:** Menggunakan **CSPRNG**.
- **Verification:**
 - Server generate $\text{ServerSeed} + \text{ClientSeed}$.
 - Server mengirim $\text{Hash}(\text{ServerSeed})$ di awal match.
 - Di akhir match, Server me-reveal ServerSeed asli.
- **Replay Consistency:** serverSeed dan roundSeed wajib digunakan ulang saat replay untuk menjamin urutan kartu dan dealer yang identik.

4.2 Dealing

- 52 Kartu, 13 per pemain, arah **Clockwise**.
- Kartu pertama diberikan ke pemain di **kiri Dealer**.

4.3 Bidding Logic (ANTI-PASS v2.5 FINAL)

4.3.1 Overview

- Bidding satu putaran (**single round**).
- Aksi: PASS atau BID (8–13).

4.3.2 Anti-Pass Enforcement (v2.5)

Server mengevaluasi tangan pemain sebelum menerima input PASS.

- **New Account Definition:** $\text{newAccount} = \text{akun umur} < 3 \text{ hari OR totalMatch} < 20$.
- **Eligibility & Override Rules:**
 - **Quota:** newAccount mendapatkan **1 override PASS per match**.
 - **Restrictions:** Override hanya berlaku di **Ranked & Public Casual**. Override **OFF** secara default di **Custom Room** (Creator dapat enable/disable).
 - **Reset:** Override reset setelah match selesai (bukan per-round).
- **Strict Logic:** Jika $\text{Layak Bid } 8 == \text{TRUE}$ (dan override habis/tidak berlaku), server **MENOLAK** input PASS.
- **Validation:** Client dilarang melakukan pre-validasi PASS. Client hanya mengirim request, Server yang menolak/menerima.

4.3.3 Main Suit Evaluation (Deterministic)

- **Eligibility:** Suit memiliki ≥ 4 kartu.

- **Tie-Breaker (Mandatory):**
 1. **TotalRank** (Higher better).
 2. **CardCount** (Higher better).
 3. **HighCardCount** (Higher better).
 4. **SuitPriority: S > H > D > C.**
- **Determinism Guarantee:** Logic ini wajib konsisten di server logic, AI evaluation, dan replay system. **Client tidak boleh menghitung ini sendiri untuk validasi.**

4.3.4 Bid 8 Eligibility

Tangan pemain **Layak Bid 8** (TRUE) jika:

1. Pemain punya ≥ 1 Main Suit.
2. $\text{MainSuit.rankTotal} \geq 40$.
3. $\text{totalHandHighCardCount} \geq 5$.

4.3.5 PASS ALL Resolution

Jika 4 pemain PASS: Default kontrak **7/7**. Declarer: Pemain pertama yang PASS.

4.4 Trump Selection

- **Actor:** Declarer.
- **Timing:** Sebelum Trick 1.
- **Immutability:** Sekali dipilih, tidak bisa diubah.

4.5 Trick Engine & Freeze Rules (v2.5 Mandatory)

4.5.1 Follow-Suit & Client Lockdown

- **Server Authority:** Server mengirim legalMask pada setiap turn_update.
 - Format: legalMask: [true, false, true, ...]
- **Client Behavior:** Client **WAJIB** men-disable kartu yang ditandai false oleh server. Client **DILARANG** menghitung follow-suit logic sendiri.
- **Error Handling:** Jika client membobol UI dan mengirim kartu illegal, Server me-reject dengan ERROR_ILLEGAL_PLAY.

4.5.2 Trick Resolve Freeze (v2.5)

- **Trigger:** Begitu kartu ke-4 diterima server.
- **Client Side:**
 1. **Freeze Input:** Client menolak input apapun.
 2. **Pause UI:** Animasi berhenti sejenak.
- **Resolution:** Client menunggu event trick_resolve dari server.
- **Unfreeze:** Client hanya melakukan reveal/animasi winner winner setelah payload trick_resolve diterima. Dilarang menduga pemenang di client.

4.5.3 Resolution Logic

- IF ada Trump: Trump tertinggi menang.

- ELSE: Kartu tertinggi dari suit Lead menang.

4.6 Hidden Trump Processing

- **Behavior:** Rank disembunyikan (isHidden=true), Suit terlihat.
- **Broadcast:** { "player": seatId, "suit": "H", "isHidden": true }.
- **Reveal Timing:** Hidden trump hanya diungkap via event trick_resolve setelah 4 kartu lengkap. Client tidak boleh mencoba menebak rank.

4.7 Scoring Mapping

Bid	Target Declarer	Target Lawan
7 (Pass All)	7	7
8	8	6
9	9	5
10	10	4
11	11	3
12	12	2
13	13	1

4.8 Round & Match Scoring

4.8.1 Overview

- **roundWinner** $\in$ {teamA, teamB}.
- **Best-Of:** 3 (First to 2 wins).
- **Max Rounds Casual:** 0 (Unlimited) - configurable.

4.8.2 Ready Prompt Logic (Casual)

Setelah Match BO3:

1. Server kirim ready_prompt (Timer 30s).
2. **Response:** READY (stay) atau Timeout (kick).
3. **Result:**
 - 4/4 Ready: New Session.
 - < 4 Players: Room Disband.

4.8.3 Persistence

- **Active Snapshot:** 50 ronde terakhir.

- **Archival:** Blob storage.

4.9 DETERMINISTIC REPLAY SYSTEM (v2.5)

Sistem replay wajib deterministik dan aman.

4.9.1 Event-Only Replay

- Replay engine **HANYA** boleh menggunakan **Event Log** sebagai sumber data.
- Dilarang menggunakan Snapshot state untuk replay (kecuali inisial state).

4.9.2 Event Ordering

- Setiap event memiliki sequenceId (uint64).
- Replay player wajib mengikuti strict event order dari server.

4.9.3 Seed-Based Determinism

Replay wajib menggunakan:

- serverSeed
- roundSeed
- shuffleSeed

Untuk memastikan: dealer, shuffle, dan trump reveal konsisten 100% dengan match asli.

4.9.4 No Calculations During Replay

Client Replay Viewer **DILARANG**:

- Menghitung legal moves.
- Menghitung trick winner ulang.
- Menduga lead suit.
- Mengolah logic hidden trump.

Aturan: Client hanya me-render data visual berdasarkan payload event yang tersimpan.

4.10 GAME LOOP & TIMING (v2.6 NEW)

Menambahkan batasan waktu server untuk memproses logic dalam siklus yang stabil.

4.10.1 Server Tick Rate

Server menggunakan Tick Scheduler untuk tugas reguler:

Tick Rate Standard: SERVER_TICK = 20 Hz (50 ms interval)

- **Responsibilities:** Mengecek turn timeout, grace countdown, dan freeze timeout.
- **AI Budgeting:** AI inference diproses per-tick untuk stabilitas CPU.

4.10.2 Turn Action Timer

Setiap player memiliki batas waktu turn: TURN_TIME_LIMIT = 25 detik.

- **Behavior:**
Jika waktu habis, server otomatis memainkan lowest legal card dari tangan pemain.
Pemain ditandai `TIMEOUT_FLAG = true`.
Jika timeout terjadi 3x dalam 1 match → Trigger AI Takeover.
- **Freeze Exemption:** Ketika freeze aktif (di luar turn pemain), `TURN_TIME_LIMIT` berhenti.

4.10.3 Freeze Timeout Fallback

Mencegah UI freeze permanen jika event resolve terlambat.

- **Threshold:** `FREEZE_MAX_WAIT` = 5 detik.
- **Client Action:** Jika `trick_resolve` tidak diterima dalam 5 detik, client wajib melakukan `GET /match/sync` dan menampilkan loading state.
- **Server Action:** Pada tick, server akan me-resend `trick_resolve` jika mendeteksi client desync.

5. AI REQUIREMENTS

5.1 Overview (Updated v2.6)

- **Telemetry:** Emit event `ai_decision`.
- **State:** `MatchState` object (Server-side only).
- **Metadata:** Setiap object AI Player wajib memiliki tier tag.
`"aiTier": "NONE" | "BASIC" | "STANDARD" | "TOURNAMENT"`
- **Required Metrics:** Setiap aksi AI harus emit:
`"aiInferTimeMs": <integer>`
`"aiPolicyVersion": "v1.x.x"`

5.2 AI Tiers

- **Tier A (Basic):** Legal, Low CPU, Follow suit priority.
- **Tier B (Standard):** Card Tracking + EV.
- **Tier C (Tournament):** MCTS + Belief State.

5.3 AI Takeover (v2.5 Rules)

- **State Inheritance:** AI menggunakan kartu asli pemain.
- **Logic:** AI melanjutkan strategi pemain sebelumnya.

6. NETWORK REQUIREMENTS

6.1 WebSocket Protocol

- **Format:** JSON.
- **Server → Client (Updated):**
 - `turn_update` (Wajib include `legalMask`).
 - `trick_resolve` (Wajib include full cards & winner untuk unfreeze UI).

- match_start, deal, bidding_update, card_played, score_update, match_end.

6.2 Ordering Guarantee

Server wajib memproses pesan secara **FIFO**.

6.3 Disconnect & Penalty

- **Grace Timer:** 60 detik.
- **Reconnect:**
 - Before 60s: Restore session.
 - After AI Played: Join sebagai **Spectator** hingga round end.
- **Penalty:** -16 MMR jika tim kalah karena DC. 3x DC dalam 24 jam = 24h Ban.

6.4 NETWORK HEALTH & LATENCY (v2.6 NEW)

6.4.1 Latency Tolerance Frame

Mencegah penalti untuk pemain dengan jaringan fluktuatif.

- **Max Allowed Client Delay:** MAX_CLIENT_DELAY = 150 ms.
Pesan yang datang dengan timestamp client <= 150ms dari batas waktu server tetap diterima.
- **Late Arrival Handling:** Jika card_play tiba setelah toleransi habis, server menolak request dan memaksa resync.

6.4.2 Network Heartbeat

Menentukan status koneksi aktif.

- **Interval:** HEARTBEAT_INTERVAL = 5 detik.
- **Logic:**
 - 3 missed heartbeats = connection_suspect.
 - 5 missed heartbeats = DISCONNECT.
- **Snapshot Trigger:** Jika player kembali dari suspect, server memaksa full state sync.

7. STATE MACHINE

WAITING_PLAYERS → DEAL → BIDDING → TRUMP_SELECTION → TRICK_LOOP (Iterasi 1–13: CARD_PLAY → TRICK_RESOLVE w/ Freeze) → ROUND_SCORE → DEALER_ROTATE → READY_PROMPT → NEXT_ROUND / MATCH_END.

8. DATA STRUCTURES (SCHEMA)

8.1 Card Object

```
{ "rank": "A..2", "suit": "S|H|D|C", "isTrump": bool, "isHidden": bool }
```


8.2 Player State (v2.6)

```
{ "playerId": "uuid", "seat": 0, "hand": [], "isConnected": true, "isAi": false, "aiTier": "STANDARD", "newAccount": bool, "passOverridesLeft": 1 }
```

8.3 Turn Update Payload (Mandatory)

```
{ "activePlayerId": "...", "leadSuit": "H", "legalMask": [true, false, true, ...] }
```

8.4 Trick Resolve Payload (Mandatory)

```
{ "cards": [...], "winner": seatId, "winningCard": {...} }
```

9. NON-FUNCTIONAL REQUIREMENTS & SECURITY (v2.6)

9.1 Performance (Updated)

- **Tick Rate:** 20 Hz (50ms).
- Latency Max 200ms.
- Concurrency 5000 matches.

9.2 Client Security (Lockdown)

- **UI-Only:** Client tidak menyimpan rule game.
- **No Logic Generation:** Client tidak boleh menentukan follow-suit, memvalidasi PASS, atau menentukan main suit.
- **Validation Authority:** Server adalah satu-satunya sumber kebenaran.

9.3 Reliability

- Redis snapshot per trick.

9.4 Client-Side Constraints (v2.6 UI Budget)

Client wajib mengikuti batas animasi untuk konsistensi replay:

- `PLAY_ANIMATION_MAX` = 400ms
- `REVEAL_ANIMATION_MAX` = 600ms
- `TRANSITION_ANIMATION_MAX` = 800ms

10. ERROR HANDLING

- **Recoverable:** Reconnect.
- **Unrecoverable:** Corrupted Deck.
- **Codes:** `ERROR_ILLEGAL_PLAY` (Client hacking attempt), `BID_FORCED_MIN_8`.

11. DISCONNECT / TIMEOUT SYSTEM

- T-0: Player DC.
- T+60s: AI Takeover.
- T < 5m: Reclaim allowed (conditional).
- T > 5m: Lockout.

12. ROOM LOGIC (Updated v2.6)

- **Public:** Auto-start 4 players. Override Pass: ON (conditional). Action Timer: ON (25s).
- **Custom:** Creator options:
 - Override Pass (ON/OFF).
 - Action Timer (ON/OFF).
 - Freeze Timeout Config (3-5s).
 - Hidden Trump Visibility (Strict/Loose).

13. MATCHMAKING SYSTEM

- **Algorithm:** Expansion Window.
- **Anti-Abuse:** Block pairing jika > X match bersama dalam 7 hari.

14. SPECTATOR SYSTEM

- **Delay:** Real-time masked atau 1 trick delay.
- **Restrictions:** No Hidden Trump / Hand view.

15. TEST PLAN

- **Unit Tests:** Anti-Pass v2.5 Logic, Tie-Breaker.
- **Integration Tests:** Full-Match Deterministic Simulation.
- **Replay Tests:** Verifikasi replay log menghasilkan visual yang 100% sama dengan live match menggunakan seed yang sama.

16. DATA & TRAINING PIPELINE

- **Event Stream:** Redis Streams.
- **ETL:** Normalize → Postgres Partitioned.

17. OPERATIONAL REQUIREMENTS

- **Scaling:** Stateless Game Server.
- **Monitoring:** Alerts (Trick process > 200ms).

18. IMPLEMENTATION ROADMAP

- Sprint 1: Core Engine & HMAC.
- Sprint 2: WebSocket & Client Lockdown (v2.5).

- Sprint 3: Matchmaking & Anti-Abuse.
- Sprint 4: AI & Replay Engine (v2.5).
- Sprint 5: Timing Hardening & Latency (v2.6).

19. PRACTICAL NOTES

- Gunakan Canonical State Hashing.
- Immutable Telemetry.

20. FUTURE EXPANSIONS

Cosmetics, Ranked Seasons, Tournament Dashboard.

21. APPENDIX A: IMPLEMENTATION REFERENCE (v2.6 UPDATED)

21.1 Config (Constants)

```
export const RANKED = { K_FACTOR: 32 };
export const ANTI_BOOST = { MAX_PAIR_IN_7D: 10, MMR_DIFF_LIMIT: 300 };
export const BID8 = { MAIN_RANK_MIN: 40, TOTAL_HC_MIN: 5 };
// v2.6 Timing
export const TIME = {
  TAKEOVER_GRACE_SEC: 60,
  SERVER_TICK_MS: 50,
  TURN_LIMIT_SEC: 25,
  FREEZE_WAIT_SEC: 5,
  HEARTBEAT_SEC: 5,
  MAX_CLIENT_DELAY_MS: 150
};
export const ANIMATION = {
  PLAY_MS: 400,
  REVEAL_MS: 600,
  TRANSITION_MS: 800
};
// v2.5 Anti-Abuse
export const NEW_ACC = { MAX_DAYS: 3, MAX_MATCHES: 20 };
```

21.2 Redis / Postgres Schema Snippets

Postgres (DDL)

```
CREATE TABLE match_log (
```

```

match_id UUID PRIMARY KEY,
server_seed TEXT NOT NULL,      -- Mandatory for Replay
round_seeds JSONB NOT NULL,     -- Mandatory for Replay
dealer_indices JSONB NOT NULL,
round_scores JSONB NOT NULL,
event_log_pointer TEXT,        -- Source of Truth for Replay
created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
ended_at TIMESTAMP WITH TIME ZONE
);

-- History perubahan rank player
CREATE TABLE player_rank_history (
  id BIGSERIAL PRIMARY KEY,
  player_id UUID NOT NULL,
  mmr_before INT NOT NULL,
  mmr_after INT NOT NULL,
  match_id UUID,
  timestamp TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- Log misconduct untuk anti-cheat/ban logic
CREATE TABLE player_misconduct_log (
  id BIGSERIAL PRIMARY KEY,
  player_id UUID NOT NULL,
  match_id UUID,
  reason VARCHAR(50),          -- e.g., 'DISCONNECT_LOSS', 'ILLEGAL_MOVE', 'TIMEOUT_STALL'
  penalty_mmr INT DEFAULT 0,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

```

21.3 evaluate_main_suit(hand) → SuitEvaluation (v2.5 Deterministic)

```

type Card = { rank: string, suit: 'S'|'H'|'D'|'C' };
type SuitEval = {
  suit: 'S'|'H'|'D'|'C',
  cardCount: number,
  highCardCount: number,
  rankTotal: number
};

const RANK_VALUE: Record<string, number> = {
  'A':14,'K':13,'Q':12,'J':11,'10':10,

```

```

    '9':9,'8':8,'7':7,'6':6,'5':5,'4':4,'3':3,'2':2
  };
  const HIGH_CARDS = new Set(['A','K','Q','J','10']);
  const SUIT_PRIORITY = { 'S': 4, 'H': 3, 'D': 2, 'C': 1 }; // Tie-breaker #4 (Higher is better)

  function evaluate_main_suit(hand: Card[]): SuitEval[] {
    const suits = { 'S': [], 'H': [], 'D': [], 'C': [] };
    hand.forEach(c => suits[c.suit].push(c));

    const evals: SuitEval[] = [];
    for (const s of ['S','H','D','C']) {
      const cards = suits[s];
      if (cards.length < 4) continue; // Eligibility check

      let rankTotal = 0;
      let highCardCount = 0;
      cards.forEach(c => {
        rankTotal += RANK_VALUE[c.rank];
        if (HIGH_CARDS.has(c.rank)) highCardCount++;
      });
      // Add length bonus if needed, but strict RankTotal is priority
      evals.push({ suit: s as any, cardCount: cards.length, highCardCount, rankTotal });
    }

    // DETERMINISTIC SORT (v2.5 Mandatory)
    // Tie-breaker order: RankTotal > CardCount > HighCardCount > SuitPriority (S>H>D>C)
    return evals.sort((a, b) => {
      if (b.rankTotal !== a.rankTotal) return b.rankTotal - a.rankTotal; // 1. TotalRank
      if (b.cardCount !== a.cardCount) return b.cardCount - a.cardCount; // 2. CardCount
      if (b.highCardCount !== a.highCardCount) return b.highCardCount - a.highCardCount; // 3.
      HC Count
      return SUIT_PRIORITY[b.suit] - SUIT_PRIORITY[a.suit]; // 4. Suit Priority (S>H>D>C)
    });
  }

```

21.4 isEligibleBid8(hand, isNewAccount, overridesLeft, isCustomRoom, overrideEnabled)

```

function isEligibleBid8(
  hand: Card[],
  isNewAccount: boolean,
  overridesLeft: number,

```

```

isCustomRoom: boolean,
overrideEnabled: boolean
): boolean {
  // v2.5 Override Logic
  let canOverride = false;

  if (isCustomRoom) {
    // Custom room depends on creator setting
    if (overrideEnabled && isNewAccount && overridesLeft > 0) canOverride = true;
  } else {
    // Public/Ranked: Always active for new accounts
    if (isNewAccount && overridesLeft > 0) canOverride = true;
  }

  if (canOverride) return false; // Allowed to pass (Soft Override)

  // Strict Check
  const evals = evaluate_main_suit(hand);
  if (evals.length === 0) return false;

  const main = evals[0];
  const totalHighCards = hand.reduce((acc, c) => acc + (HIGH_CARDS.has(c.rank) ? 1 : 0), 0);

  if (main.rankTotal < BID8.MAIN_RANK_MIN) return false;
  if (totalHighCards < BID8.TOTAL_HC_MIN) return false;
  return true;
}

```